Phage SECURITY

Legend.trade

Security Review

Conducted by:

Pyro, Security Researcher
YanecaB, Security Researcher

28.10.2025

Table of Contents

Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

Legend.trade is a competitive perpetual trading platform where users compete in team based duels or multiplayer tournaments for prizes. Players trade through isolated proxy accounts on the GTE perpetual exchange, with the best performing traders (by ROI or PnL) winning the bet duels or tournament prizes.

Executive summary

Overview

Project Name	Legend.trade
Repository	private
Commit hash	20bbec47c7b3f5b46511785b6c9871ab8f097dd3
Remediation	4e6f542529aa284cbc9c91f749d7e163e7a1a4ee
Methods	Manual review

Timeline

Audit kick-off	23.10.2025
End of audit	26.10.2025
Remediations start	27.10.2025
Remediations end	27.10.2025

Scope

DuelManager.sol

PlayerProxyBeacon.sol

PlayerProxyFactory.sol

PlayerProxyGTE.sol

Tournament.sol

Issues Found

Severity	Count
High	0
Medium	2
Low	2

Findings

Medium Severity

[M-01] Users can withdraw before a tournament is settled

Users can withdraw before a tournament is settled:

```
    function withdrawFromTournament(uint256 tournamentId, uint256 amount)
       external {
        // ...
        bool tournamentFailed = block.timestamp >=
           tournament.config.registrationEnd
            && tournament.participants.length() <
               tournament.config.minParticipants;

        uint256 tradingEnd = tournament.config.tradingStart +
           COUNTDOWN_DURATION + tournament.config.tradingDuration;
        bool tournamentEnded = block.timestamp >= tradingEnd;
```

This will cause their final **PnL** to be lowered due to us removing `sessionBudget` from their account value:

```
            Participant storage p = tournament.participantData[player];

            // Calculate PnL from proxy
            IPlayerPlatformProxy proxy = IPlayerPlatformProxy(payable(p.proxy));
            int256 finalValue = proxy.getAccountValue();

            pnls[i] = finalValue - int256(tournament.config.sessionBudget);
```

Recommendation

Make sure no user can call `withdrawFromTournament` if the tournament is started, but not settled.

```
     uint256 tradingEnd = tournament.config.tradingStart + COUNTDOWN_DURATION
        + tournament.config.tradingDuration;
     bool tournamentEnded = block.timestamp >= tradingEnd;

     if (!tournamentFailed && !tournamentEnded) revert CannotWithdraw();

+      if (tournamentEnded && !tournament.settled) revert TournamentNotSettled(
    );
```

Resolution: Fixed in 0a3558d

[M-02] Duel manager can go insolvent

`DuelManger` can go insolvent due to a rounding error inside `acceptDuel`.

```
        if (isInTeamA) {
            betPerMember = duel.bet / teamA.members.length();
        } else {
            betPerMember = duel.bet / teamB.members.length();
        }
```

The issue appears because the bet can be lower than the team count, resulting in `betPerMember` in one of the teams being 0.

If the other team wins, they would get their winnings from the contract balance. This will make the contract insolvent, and if all bets are closed and claimed, the last winner will not be able to claim their winnings as the contract will try to send funds that it doesn't have.

Example:

1. `teamA` is only Alice, `teamB` are 5 other addresses owned by Alice
2. bet is 4 wei, the rest of the config variables don't matter
3. When accepting, Alice will pay 4 wei, but everyone from `teamB` will pay 0
4. When Alice wins, her payout will be `4 * 2 = 8` wei, but the total bet for this duel was 4 wei

Recommendation

Either make sure when taking the bets it's rounded up, or have a `totalAmount` and the last winner to claim what's left from it

Resolution: Fixed in f603329

Low Severity

[L-01] There is no limit for tournament size

When making a tournament there are a few checks for participants, however there is no check for the max allowed participants inside the tournament.

```
if (config.tradingDuration == 0) revert InvalidConfiguration();
if (config.minParticipants < 2) revert InvalidConfiguration();
if (config.maxParticipants == 0) revert InvalidConfiguration();
if (config.maxParticipants < config.minParticipants) {
   revert InvalidConfiguration();
}
```

The gas limit for megaETH is 1 bill, so the amount of users that can participate should not be a problem. However in rare cases, or if a malicious user spams addresses it has the potential to DOS a tournament, as when closing one we do gas heavy operations (for loops and sorting).

Recommendation

Consider having a max limit of 1000 participants.

Resolution: Fixed in 997e702

[L-02] No check for `maxTeamSize` in `createChallenge`

The `createChallenge` function in the `DuelManager` contract fails to validate team sizes against the `maxTeamSize` variable, unlike the `createTeam` function which does. Breaking the documented contract rules.

Recommendation

Add a validation check in `createChallenge` to ensure that the lengths of both `teamA` and `teamB` arrays do not exceed `maxTeamSize`.

Resolution: Fixed in 39a5c28